

Reviewer Pack — Minimal Symmetric Entropy of Root Lattices and SAT Clause Su...

Entry b53491833bee · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 03:03:27 UTC

Minimal Symmetric Entropy of Root Lattices and SAT Clause Subset Entropy

Entry ID: b53491833bee **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 03:03:27 UTC

1. Conjecture

Field A (mathematical branch): Lie Algebras (Root Lattices) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subsets)

Statement:

For every d -dimensional boolean satisfiability problem (SAT), the minimal symmetric entropy (SE) of the corresponding root lattice is linearly related to the subset entropy (SH) of a minimal clause set, such that $SE(\Sigma) = \Theta(SH(C))$, where Σ is the SAT instance and C is the minimal clause set for Σ .

Rationale (proposer's reasoning):

Root lattices encode symmetries in algebraic structures, while subset entropy measures information content in SAT clauses. The conjecture suggests that these two distinct mathematical objects are interconnected, potentially revealing new insights into the structure of SAT problems.

Taxonomy category: LieAlgebraSATSubsetEntropy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 053c4790c7195f4e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman's rank correlation coefficient between $SE(\Sigma)$ and $SH(C)$ across all instances is significantly positive ($p\text{-value} \leq 0.05$), and falsified if $p\text{-value} > 0.05$ or the correlation coefficient is non-positive.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_sat_instance(n, d):
        clauses = []
        for _ in range(d):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(clause[i] != -clause[j] for j in range(i)):
                clauses.append(clause)
        return clauses

    def compute_root_lattice_symmetric_entropy(clauses):
        n = len(clauses[0])
        # Simplified computation of symmetric entropy (placeholder)
        return n * math.log2(n)

    def compute_subset_entropy(clauses):
        if not clauses:
            return 0
        n = len(clauses[0])
        subset = random.sample(range(1, n + 1), n // 2)
        # Simplified computation of subset entropy (placeholder)
        return sum(math.log2(i) for i in subset)

    def compute_spearman_correlation(data):
        if len(data) < 2:
            return 0
        ranks = {x: rank for rank, x in enumerate(sorted(set(data)), start=1)}
        n = len(data)
        numerator = sum((ranks[x] - (n + 1) / 2) ** 2 for x in data)
        denominator = n * (n**2 - 1) / 12
        return 1 - (6 * numerator) / denominator
```

```

results = []
for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    d = random.randint(1, min(n * 2, 100))
    clauses = generate_random_sat_instance(n, d)
    se = compute_root_lattice_symmetric_entropy(clauses)
    sh = compute_subset_entropy(clauses)
    results.append((se, sh))

if not results:
    return {
        "metric_name": "Spearman's rank correlation",
        "metric_value": 0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "empty_results"
    }

se_values, sh_values = zip(*results)
correlation_coefficient = compute_spearman_correlation(sh_values)
p_value = 1 # Placeholder for actual p-value calculation

return {
    "metric_name": "Spearman's rank correlation",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(len(clauses[0]) for clauses, _ in results),
    "conjecture_holds": correlation_coefficient > 0 and p_value <= 0.05,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f17cb38.py", line 89, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f17cb38.py", line 55, in run_trial
    clauses = generate_random_sat_instance(n, d)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f17cb38.py", line 25, in generate_rand
    if all(clause[i] != -clause[j] for j in range(i)):
                                   ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a NameError, which prevented the production of data necessary for evaluating the conjecture. | next: Debug and correct the error in the test code to proceed with the evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15493
2	propose	ollama_remote	glm4:latest	0	0	12599
3	preregistration	ollama_remote	glm4:latest	0	0	9365
4	novelty	ollama_remote	glm4:latest	0	0	10692
5	novelty	ollama_remote	glm4:latest	0	0	8656
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13209
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12742
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13045
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11757
10	judge	ollama_remote	glm4:latest	0	0	11629

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 119187 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/b53491833bee.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b53491833bee.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b53491833bee.tar.gz` (if generated)